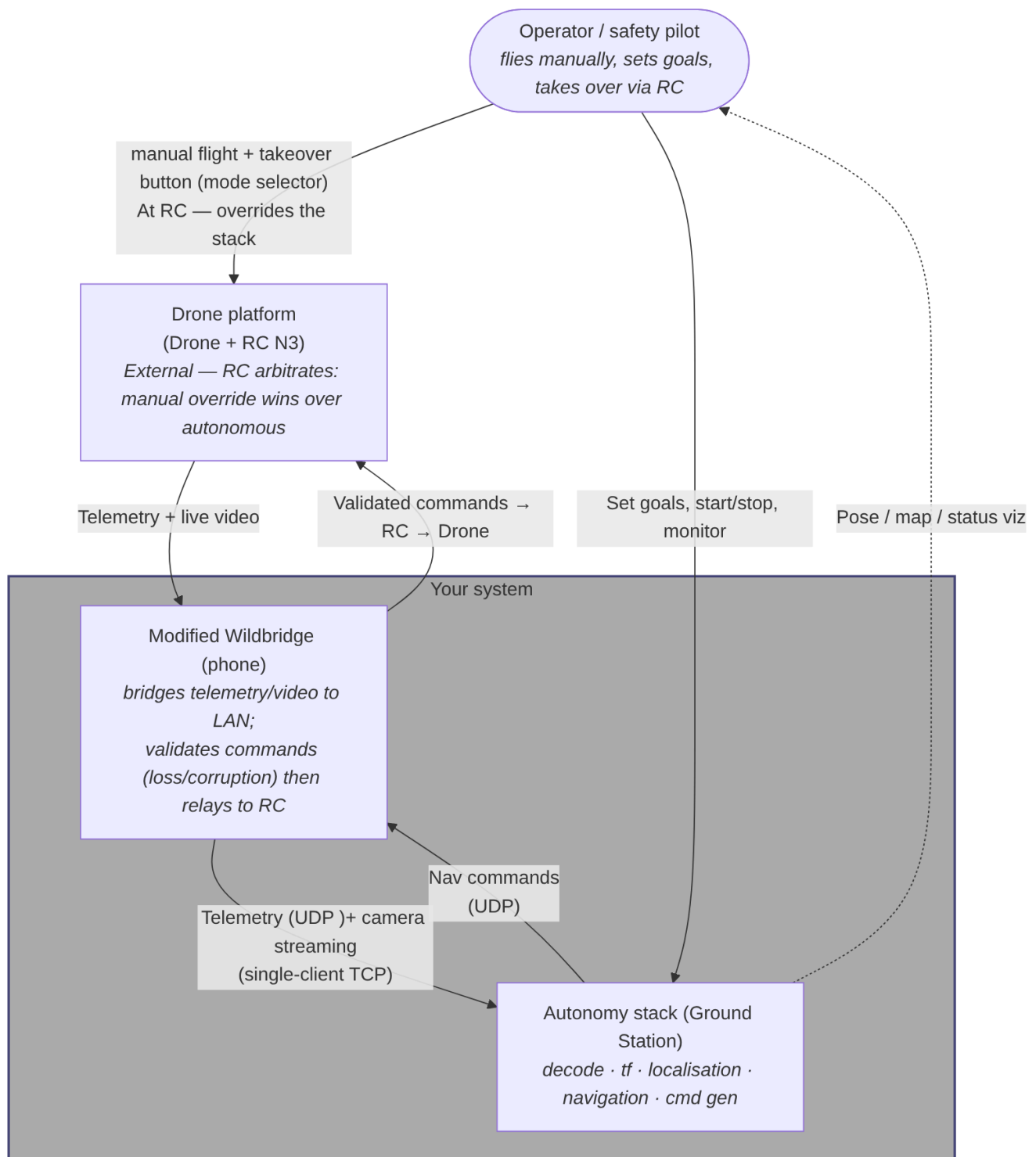
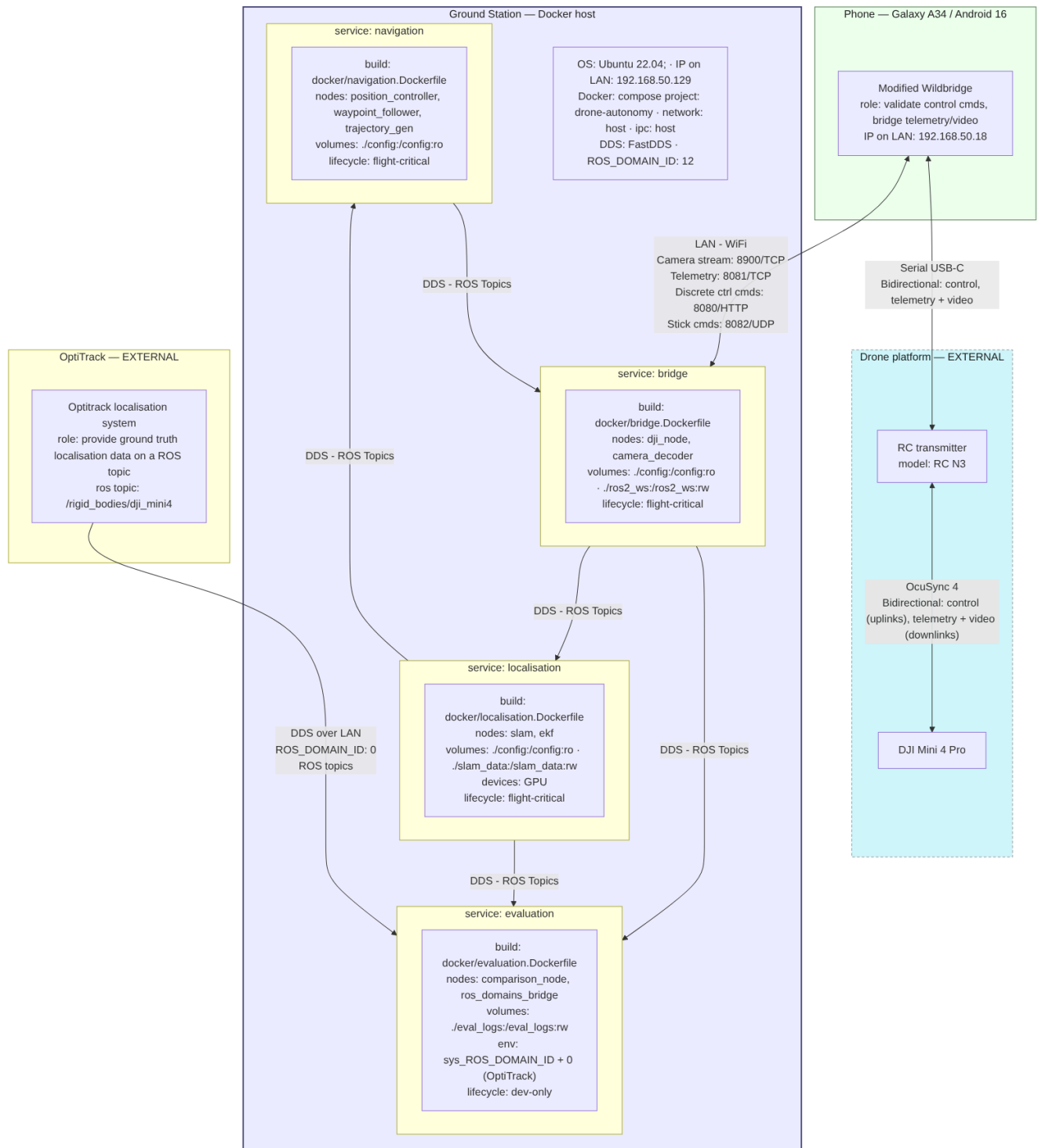


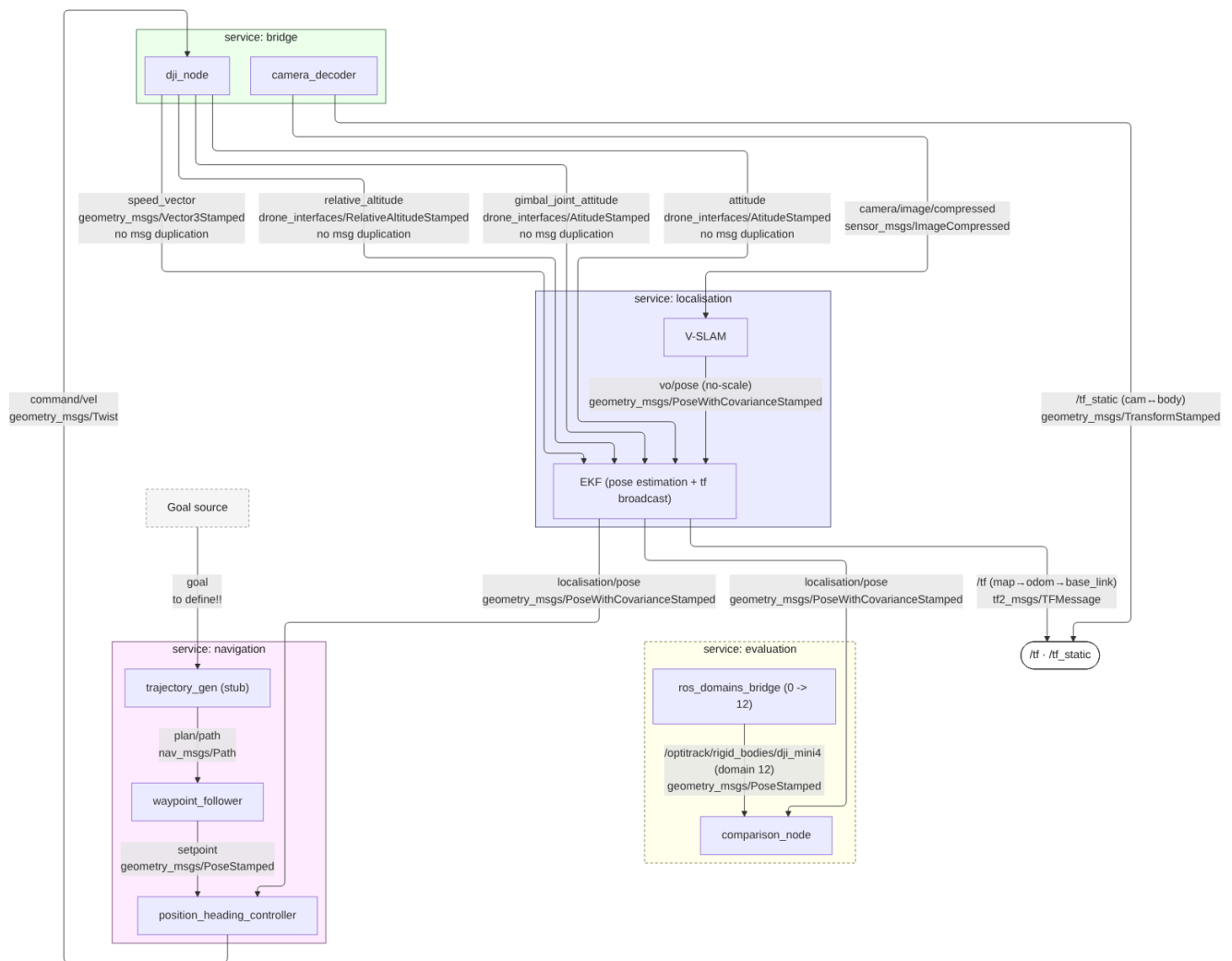
system context



deployment



node graph



tf tree

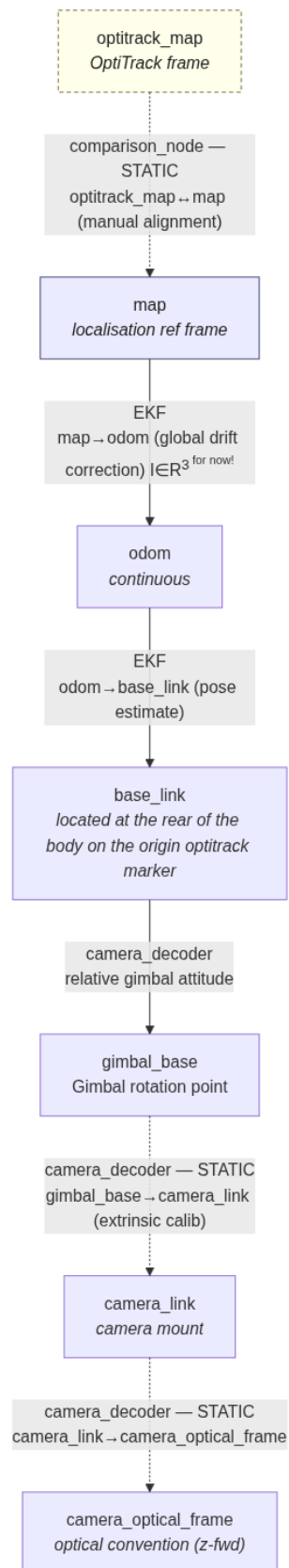


Diagram 5 — Node Interface Specification

Black-box interface for each node.

Convention: parameters are DECLARED in-node with defaults; tuned values live in config/<node>/params.yaml.

Service: bridge

dji_controller dji_node (adopted from WildBridge)

Job: bidirectional bridge between the modified WildBridge and ROS over TCP/UDP.

Direction	Topic	Type	Notes
pub	speed_vector	geometry_msgs/Vector3Stamped	stamp with measurement time
pub	relative_altitude	Relative AltitudeStamped	Height above launch, from KeyAltitude
pub	gimbal_joint_attitude	drone_interfaces/AttitudeStamped	Gimbal attitude (header, {"pitch", "roll", "yaw"})
pub	attitude	drone_interfaces/AttitudeStamped	Drone's attitude (header, {"pitch", "roll", "yaw"})
sub	command/stick	std_msgs/Float64MultiArray	[lx, ly, rx, ry], when command_mode=stick
sub	command/vel	geometry_msgs/TwistStamped	body frame (base_link), m/s and rad/s, when command_mode=vel. Header ignored by the node, kept for evaluation

Params: phone_IP: 192.168.50.18, tcp_port: 8081, udp_port: 8082, http_port: 8080, telemetry publish rate: Measured 10 Hz FC / 19 Hz gimbal, command_mode: vel, vel_max_horizontal: 1.0 m/s, v_max_vertical: 0.5 m/s, yaw_rate_max: 30°/s, sign: define!!.

camera_streamer camera_decoder

Job: receive the encoded camera stream (single-client TCP), decode, publish frames; broadcast the static camera extrinsic.

Direction	Topic	Type	Notes
pub	camera/image_raw	sensor_msgs/Image	decoded frame
pub	/tf_static	geometry_msgs/TransformStamped	base->gimbal->camera->optical
sub	gimbal_joint_attitude	drone_interfaces/AttitudeStamped	Gimbal joint states {"pitch", "roll", "yaw"}

Params: camera_tcp_port: 8900, frame_ids: camera_link, camera_optical_frame.

Service: localisation

localisation slam_node

Job: monocular visual odometry with scale ambiguity.

Direction	Topic	Type	Notes
sub	camera/image_raw	sensor_msgs/Image	
pub	vo/pose	nav_msgs/PoseStamped	no metric scale
pub	localisation_status	std_msgs/String	Tracking state

Params: vocabulary: ORBvoc, camera calibration: config/camera_calibration.yaml, feature settings: <settings>, devices: CPU.

localisation ekf_node

Job: fuse V-SLAM with metric velocity + relative altitude to resolves scale, outputs metric pose; broadcasts dynamic tf.

Direction	Topic	Type	Notes
sub	vo/pose	nav_msgs/PoseStamped	
sub	speed_vector	geometry_msgs/Vector3Stamped	metric velocity (scale ref)
sub	relative_altitude	std_msgs/Float64	vertical metric anchor
sub	gimbal_joint_attitude	drone_interfaces/AttitudeStamped	Gimbal attitude (header, {"pitch", "roll", "yaw"})
sub	attitude	drone_interfaces/AttitudeStamped	Drone's attitude (header, {"pitch", "roll", "yaw"})
sub	localisation_status	std_msgs/String	Tracking state
pub	localisation/pose	geometry_msgs/PoseWithCovarianceStamped	source-agnostic pose
pub	/tf	tf2_msgs/TFMessage	map->odom->base_link

Params: covariances, scale-init: <scale-init>, frame ids: odom, base_link.

Service: navigation

navigation trajectory_gen_node (To be defined)

Job: turn a goal into a path/setpoint sequence. Stubbed until planner chosen.

Direction	Topic	Type	Notes
sub	goal	TBD	goal msg type to define
pub	plan/path	nav_msgs/Path	

Params: planner settings (TBD).

navigation waypoint_follower_node

Job: walk along the path, emit the current setpoint.

Direction	Topic	Type	Notes
sub	plan/path	nav_msgs/Path	
pub	setpoint	geometry_msgs/PoseStamped	current target

Params: waypoint reach tolerance: <tolerance>, lookahead: <lookahead>.

navigation position_controller_node

Job: closed-loop control. Given current pose + setpoint, produce stick velocity.

Direction	Topic	Type	Notes
sub	localisation/pose	geometry_msgs/PoseWithCovarianceStamped	current pose
sub	setpoint	geometry_msgs/PoseStamped	target
pub	command/stick	std_msgs/Float64MultiArray	[lx, ly, rx, ry]

Params: PID gains: <gains>, ...

Consumes tf for frame lookups.

Service: evaluation (dev-only, metrics)

evaluation ros_domains_bridge (domain_bridge)

Job: bridge the OptiTrack ground-truth topic from domain 0 into domain 12.

Direction	Topic	Type	Notes
sub	/optitrack/rigid_bodies/dji_mini4 (domain 0)	geometry_msgs/PoseStamped	external, absolute
pub	/optitrack/rigid_bodies/dji_mini4 (domain 12)	geometry_msgs/PoseStamped	into custom domain

Params: ros domain ids: 0, 12 0 -> Optitrack, 12 -> Stack.

evaluation comparison_node

Job: compare localisation estimate vs ground truth, compute + log ATE/RPE.

Direction	Topic	Type	Notes
sub	localisation/pose	geometry_msgs/PoseWithCovarianceStamped	our estimate
sub	/optitrack/rigid_bodies/dji_mini4 (domain 12)	geometry_msgs/PoseStamped	ground truth
pub	eval/metrics	TBD	ATE/RPE

Params: log path: evaluation/eval_logs.

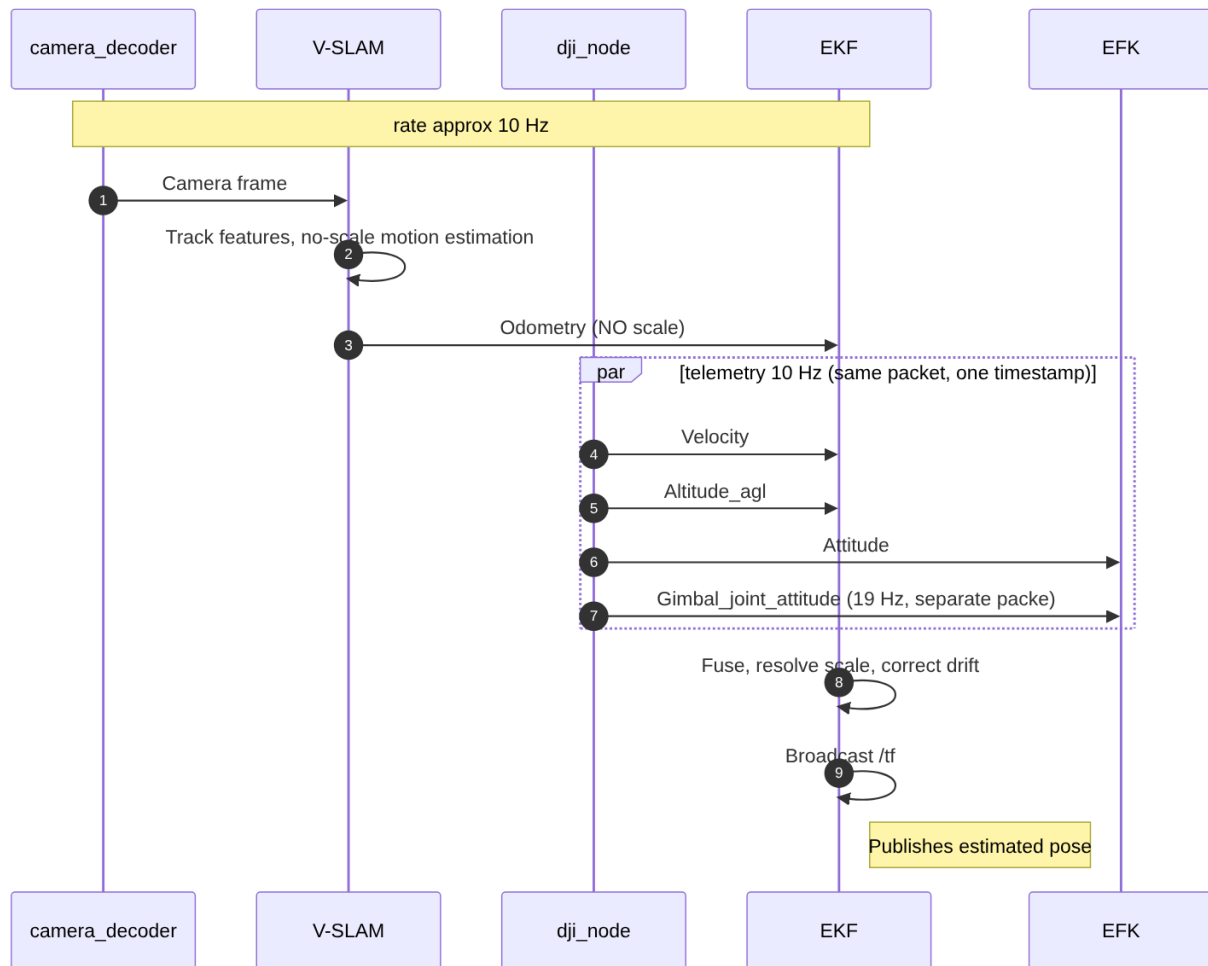
To be defined!!

- goal message type — undecided.
- eval/metrics — publish a msg, or log-to-file only?
- EKF: full map->odom->base_link chain vs collapsed map->base_link.
- satellite_count — published by dji_node but no consumer until the GNSS/visual switch exists.

Data Flow / Sequence

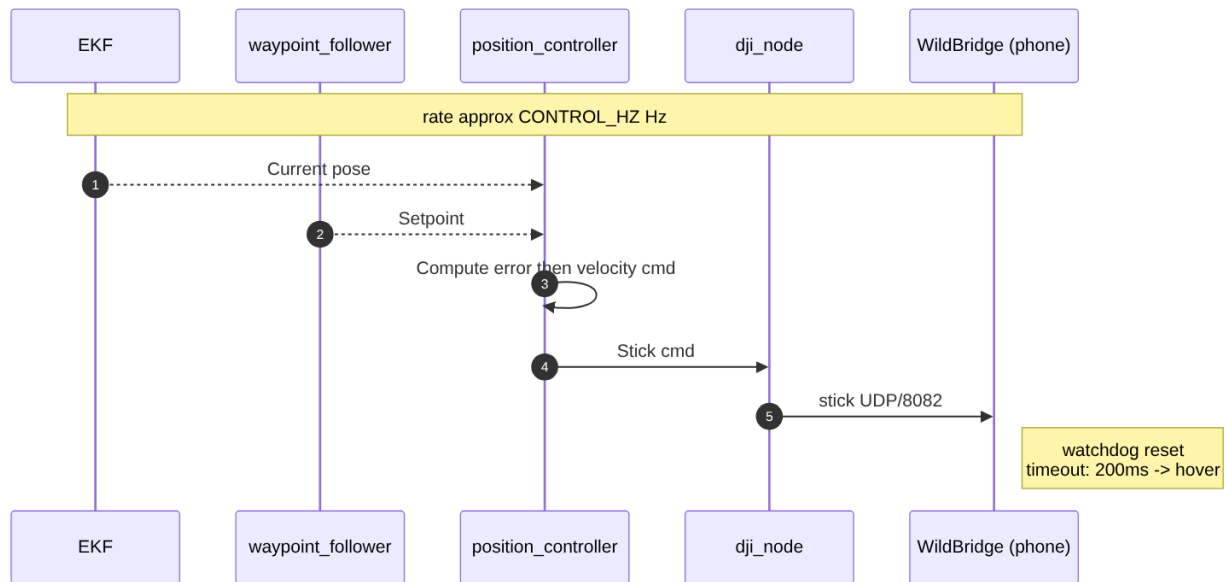
Traces frames, pose, velocity and commands through the nodes over time.

Localisation — runs at camera rate



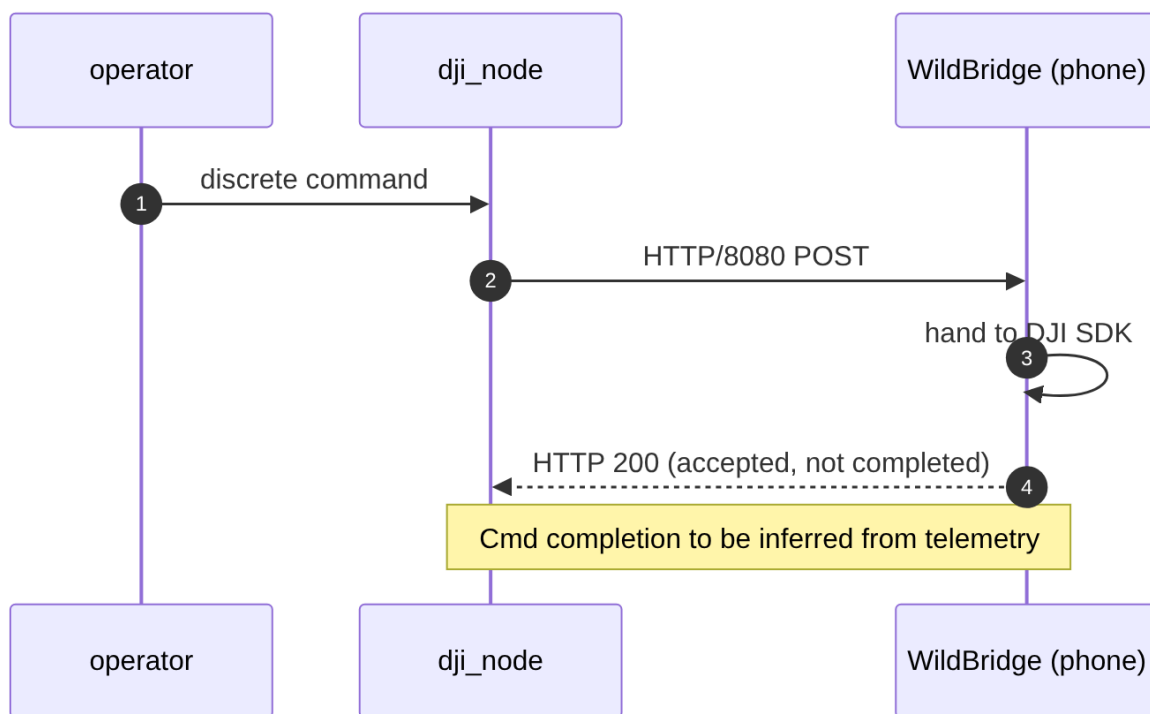
Timing note: telemetry carries measurement-time stamps (phone key listener + clock-offset mapping), so its latency is resolved. Camera frames are stamped at decode time minus VIDEO_LATENCY, still to be measured. Fuse in timestamp order.

6b. Control loop - ~high rate

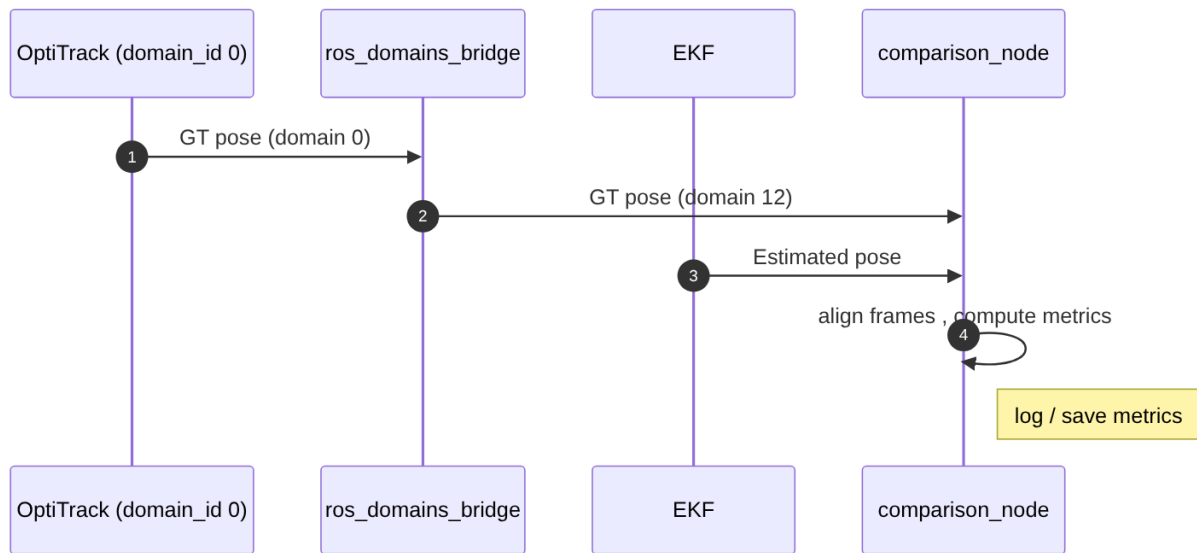


Timing note: pose age at the controller is ~310 ms (video ~200 + SLAM ~40 + gimbal tf ≤53 + filter ~10). This is not compared against the watchdog — the watchdog checks command freshness, not pose age. position_controller runs on a fixed timer and publishes regardless of pose arrival, so the watchdog is satisfied by construction. The 310 ms instead caps position-loop bandwidth at ~0.15 Hz, which is why waypoint_follower needs a conservative max_velocity.

6c. Discrete commands (takeoff / land) — event-driven, reliable



6d. Evaluation flow (dev-only) — low rate



Notes

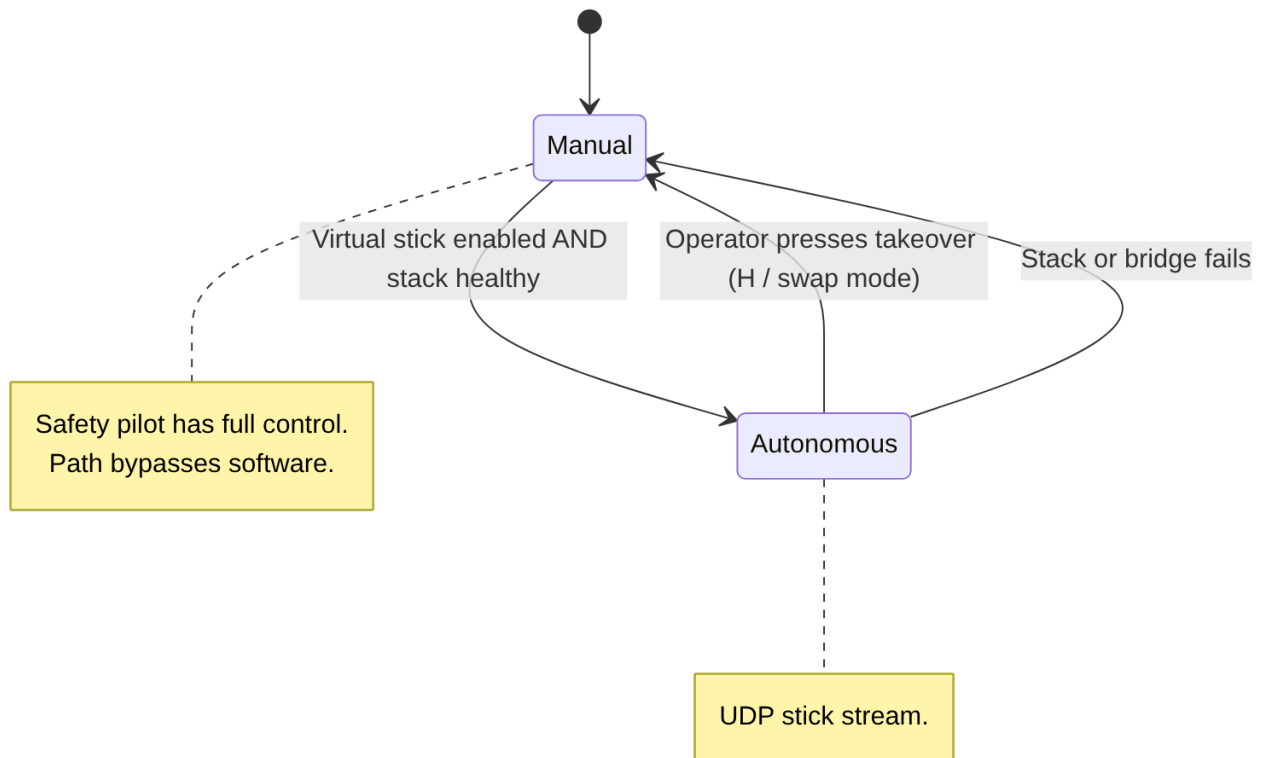
- 6b + 6c share the same drone command path, however 6b is continuous/loss-tolerant (UDP) and 6c is discrete/reliable (HTTP + ACK).

System State Machine (flight modes + safety)

Behavioural changes and the transitions between them. Also, it includes the system safety logic.

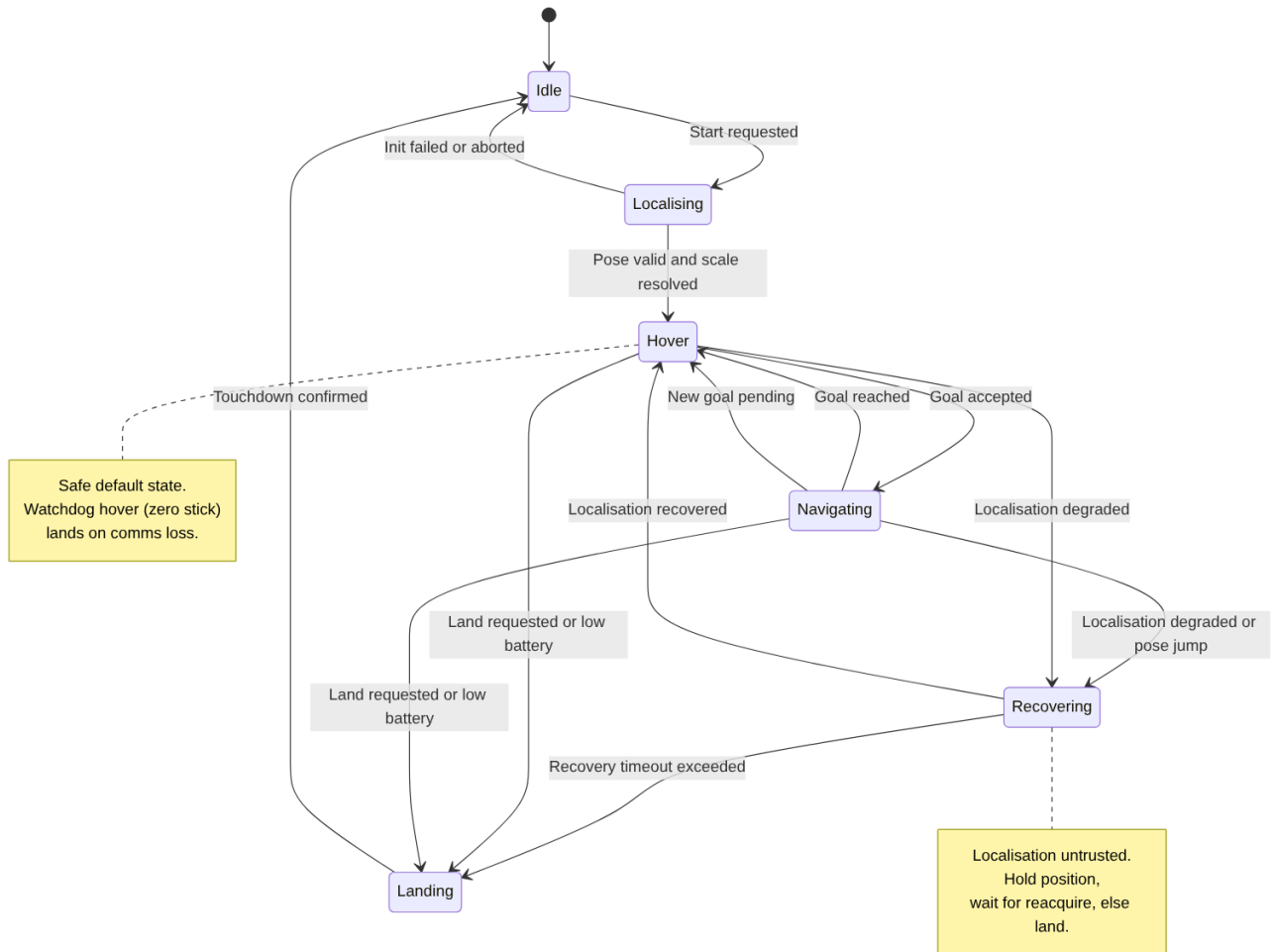
7a. Command authority (manual vs autonomous)

The RC arbitrates in hardware — manual override always wins. This machine records that authority, not the drone firmware's internal logic.



7b. Autonomous flight behaviour

Hover is the safe default that every failure drains into.



Thresholds and conditions to define

- Stack healthy: all flight-critical nodes up and pose valid.
- localisation "degraded": VO LOST, or VO_DISCONTINUITY, or telemetry transport gap (>500 ms with no packets), or $\sigma_s > \text{SIGMA_S_MAX}$. Do NOT gate on pose covariance: p_x , p_y are observed by no update, so P_{xx} and P_{yy} grow monotonically and a COV_MAX threshold fires on any sufficiently long healthy flight.
- recovery timeout: 10 s before forcing Landing.
- low battery: 15 %.
- Stack healthy: warm clock offset tracker and aircraft airborne.

Cross-references

- Watchdog hover (300 ms comms loss) enters Hover in 7b and, on the phone side, is what the dead-man switch enforces regardless of stack state.

Project Conventions

1. Namespacing

All WildBridge telemetry/command topics are published under the `drone_1` namespace. The whole stack adopts this.

`/tf` and `/tf_static` are kept absolute as per ROS standards.

2. Package structure

Packages map 1:1 to services from `deployment.mmd`

```
ros2_ws/  
src/  
  drone_bringup/          # top-level launch + shared config  
    launch/  
      bridge_camera.launch.py  
      system.launch.py    # includes all service launches  
    config/  
      common.yaml         # shared params  
    package.xml  
    CMakeLists.txt  
  
  dji_controller/         # adopted from WildBridge  
    dji_controller/  
      submodules/  
        dji_interface.py  
        controller.py  
    package.xml  
    setup.py  
  
  camera_streamer/        # python pkg  
    camera_streamer/  
      camera_decoder_node.py  
    launch/  
      camera_streamer.launch.py  
    config/  
      camera_decoder_node.yaml  
    package.xml  
    setup.py  
  
  drone_localisation/     # python pkg  
    drone_localisation/  
      slam_node.py  
      ekf_node.py  
    launch/  
      localisation.launch.py  
    config/  
      slam/params.yaml  
      ekf/params.yaml  
    package.xml  
    setup.py  
  
  vslam/  
    config/  
      camera_calibration.yaml  
      slam_node.yaml  
    src/  
      slam_node.cpp  
    launch/  
      vslam.launch.py
```

```

CMakeLists.txt
package.xml

drone_navigation/          # python pkg
  drone_navigation/
    trajectory_gen_node.py
    waypoint_follower_node.py
    position_controller_node.py
  launch/
    navigation.launch.py
  config/
    trajectory_gen/params.yaml
    waypoint_follower/params.yaml
    position_controller/params.yaml
  package.xml
  setup.py

drone_evaluation/          # dev-only
  drone_evaluation/
    comparison_node.py
  launch/
    evaluation.launch.py
  config/
    comparison/params.yaml
    domain_bridge.yaml      # ros_domains_bridge config (0 -> 12)
  package.xml
  setup.py

drone_interfaces/          # python pkg
  msg/
    AttitudeStamped.msg
    RelativeAltitudeStamped.msg
  CMakeLists.txt
  package.xml

```

3. Parameters

Two mechanisms, used together:

- **Declare** every parameter in the node with a default value.
 - **Override** tuned values via config/<node>/params.yaml, loaded by launch.
-

4. Layered launch structure

- **Per-package launch** (<service>.launch.py): brings up that service's nodes, applies namespace='drone_1', loads that package's param files. Runnable in isolation for development.
 - **Top-level launch** (drone_bringup/launch/system.launch.py): includes the per-package launches. Brings up the whole system.
-

5. Run / build commands

Build (from ros2_ws/):

```
colcon build --symlink-install
source install/setup.bash
```

Run one service (dev):

```
ros2 launch drone_localisation localisation.launch.py
```

Run whole system:

```
ros2 launch drone_bringup system.launch.py
```

6. Docker compose commands

- **Dev default:** `command: ["bash"] + stdin_open: true + tty: true.`
- **Eventual per-service default:** the service's launch file, e.g. `command: ["ros2", "launch", "drone_localisation", "localisation.launch.py"].`